ASSIGNMENT - 11.3

Lab 11: Data Structures with AI Implementing Fundamental Data

Structures using AI Assistance

NAME: Y.sahasra

HTNO: 2303A51269

BTNO:19

Task 1: Smart Contact Manager (Arrays & Linked Lists)

Scenario

SR University's student club requires a simple Contact Manager Application to store members' names and phone numbers. The system should support efficient addition, searching, and deletion of contacts.

Tasks

1. Implement the contact manager using arrays (lists).
2. Implement the same functionality using a linked list for dynamicmemory allocation.
3. Implement the following operations in both approaches: o Add a contact o Search for a contact o Delete a contact
4. Use GitHub Copilot to assist in generating search and delete methods.
5. Compare array vs. linked list approaches with respect to:

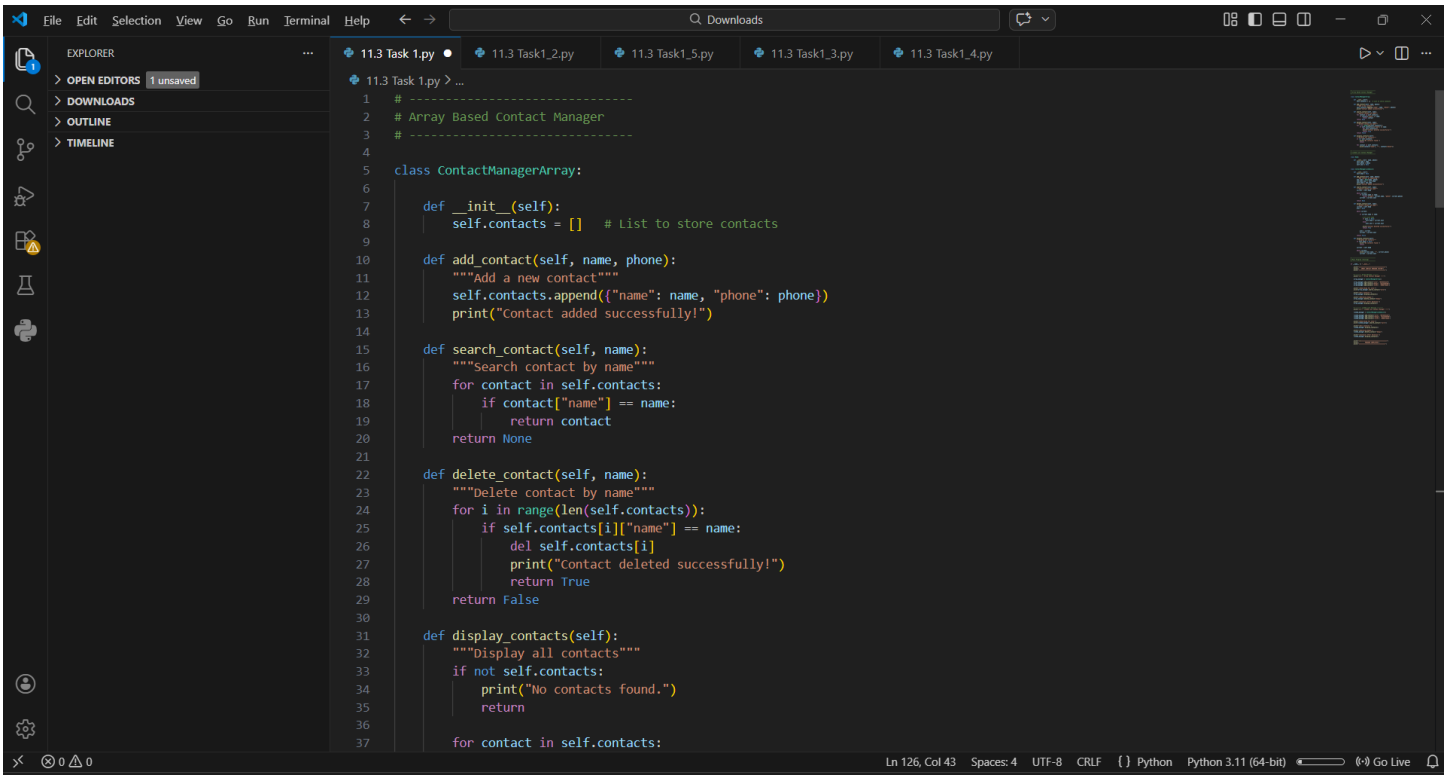
o Insertion efficiency o

Deletion efficiency

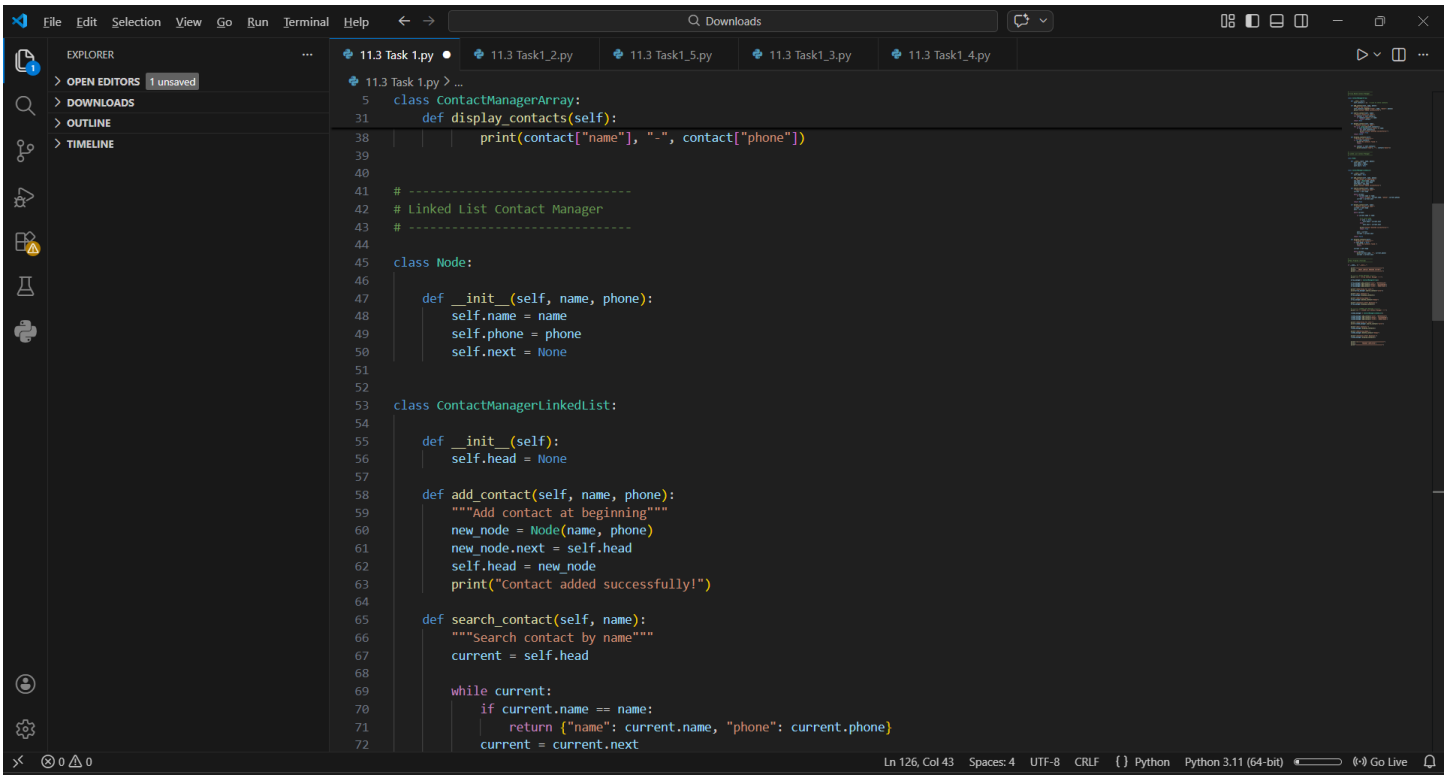
Expected Outcome

- Two working implementations (array-based and linked-list-based).
- A brief comparison explaining performance differences.

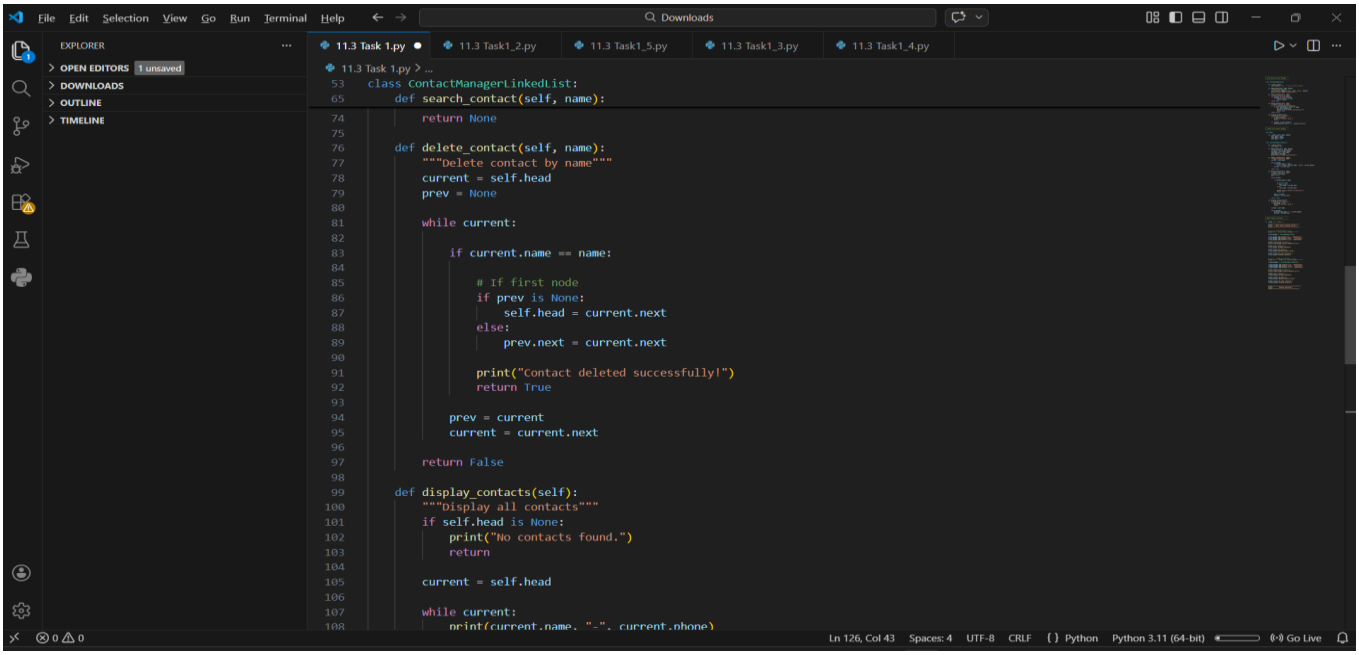
CODE:



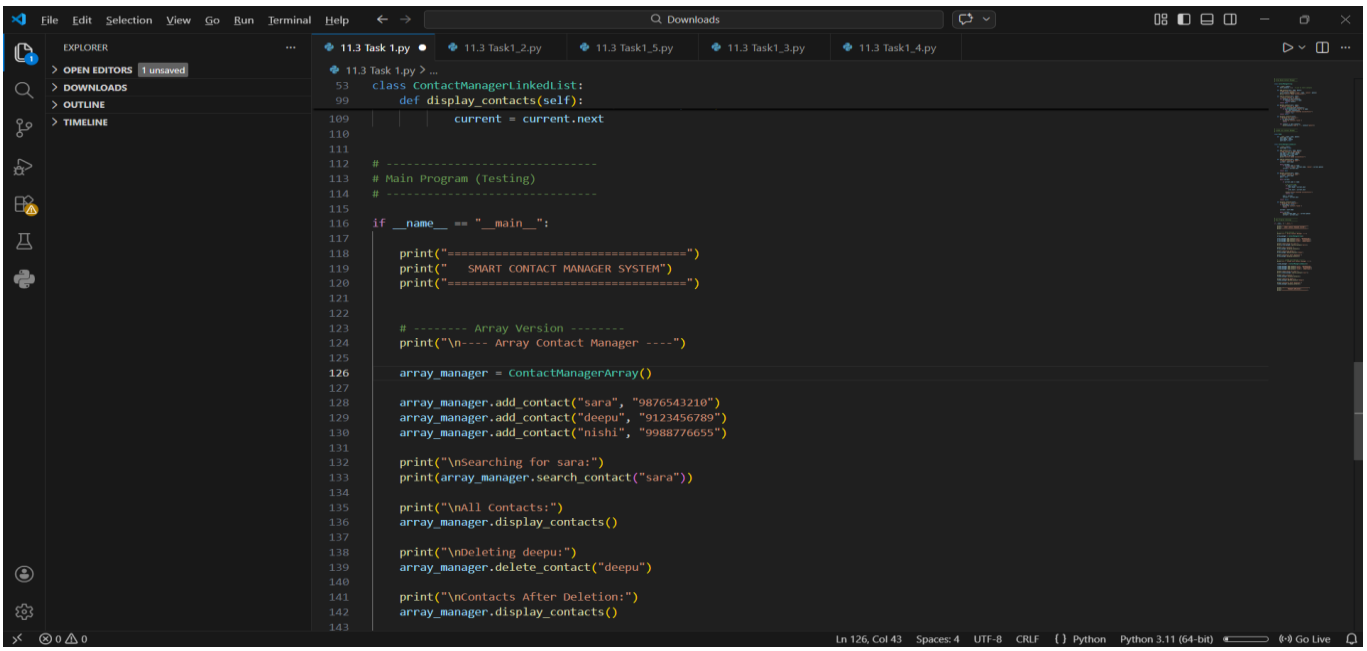
```python
# ------------------------------
# Array Based Contact Manager
# ------------------------------

class ContactManagerArray:

    def __init__(self):
        self.contacts = []   # List to store contacts

    def add_contact(self, name, phone):
        """Add a new contact"""
        self.contacts.append({"name": name, "phone": phone})
        print("Contact added successfully!")

    def search_contact(self, name):
        """Search contact by name"""
        for contact in self.contacts:
            if contact["name"] == name:
                return contact
        return None

    def delete_contact(self, name):
        """Delete contact by name"""
        for i in range(len(self.contacts)):
            if self.contacts[i]["name"] == name:
                del self.contacts[i]
                print("Contact deleted successfully!")
                return True
        return False

    def display_contacts(self):
        """Display all contacts"""
        if not self.contacts:
            print("No contacts found.")
            return

        for contact in self.contacts:
```



```python
class ContactManagerArray:
    def display_contacts(self):
                print(contact["name"], "-", contact["phone"])


# ------------------------------
# Linked List Contact Manager
# ------------------------------

class Node:

    def __init__(self, name, phone):
        self.name = name
        self.phone = phone
        self.next = None


class ContactManagerLinkedList:

    def __init__(self):
        self.head = None

    def add_contact(self, name, phone):
        """Add contact at beginning"""
        new_node = Node(name, phone)
        new_node.next = self.head
        self.head = new_node
        print("Contact added successfully!")

    def search_contact(self, name):
        """Search contact by name"""
        current = self.head

        while current:
            if current.name == name:
                return {"name": current.name, "phone": current.phone}
            current = current.next
```

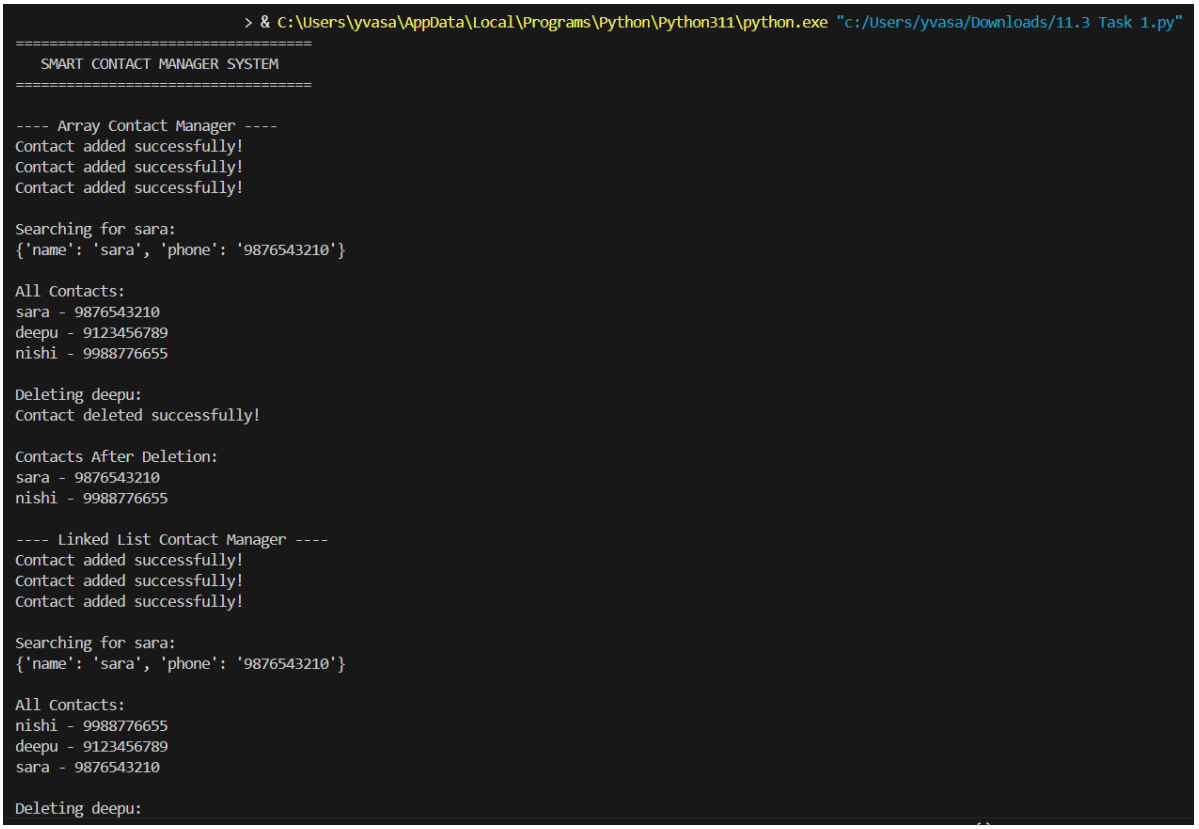


```
class ContactManagerLinkedList:
    def search_contact(self, name):
        return None

    def delete_contact(self, name):
        """Delete contact by name"""
        current = self.head
        prev = None

        while current:

            if current.name == name:

                # If first node
                if prev is None:
                    self.head = current.next
                else:
                    prev.next = current.next

                print("Contact deleted successfully!")
                return True

            prev = current
            current = current.next

        return False

    def display_contacts(self):
        """Display all contacts"""
        if self.head is None:
            print("No contacts found.")
            return

        current = self.head

        while current:
            print(current.name, "-", current.phone)
```



```
class ContactManagerLinkedList:
    def display_contacts(self):
            current = current.next


# --------------------------------
# Main Program (Testing)
# --------------------------------

if __name__ == "__main__":

    print("==================================")
    print("   SMART CONTACT MANAGER SYSTEM")
    print("==================================")


    # -------- Array Version --------
    print("\n---- Array Contact Manager ----")

    array_manager = ContactManagerArray()

    array_manager.add_contact("sara", "9876543210")
    array_manager.add_contact("deepu", "9123456789")
    array_manager.add_contact("nishi", "9988776655")

    print("\nSearching for sara:")
    print(array_manager.search_contact("sara"))

    print("\nAll Contacts:")
    array_manager.display_contacts()

    print("\nDeleting deepu:")
    array_manager.delete_contact("deepu")

    print("\nContacts After Deletion:")
    array_manager.display_contacts()
```

OUTPUT:



Task 2: Library Book Search System (Queues & Priority Queues)

Scenario

The SRU Library manages book borrow requests. Students and faculty submit requests, but faculty requests must be prioritized over student requests.

Tasks

1. Implement a Queue (FIFO) to manage book requests.
2. Extend the system to a Priority Queue, prioritizing faculty requests.
3. Use GitHub Copilot to assist in generating:

o enqueue() method o

dequeue() method

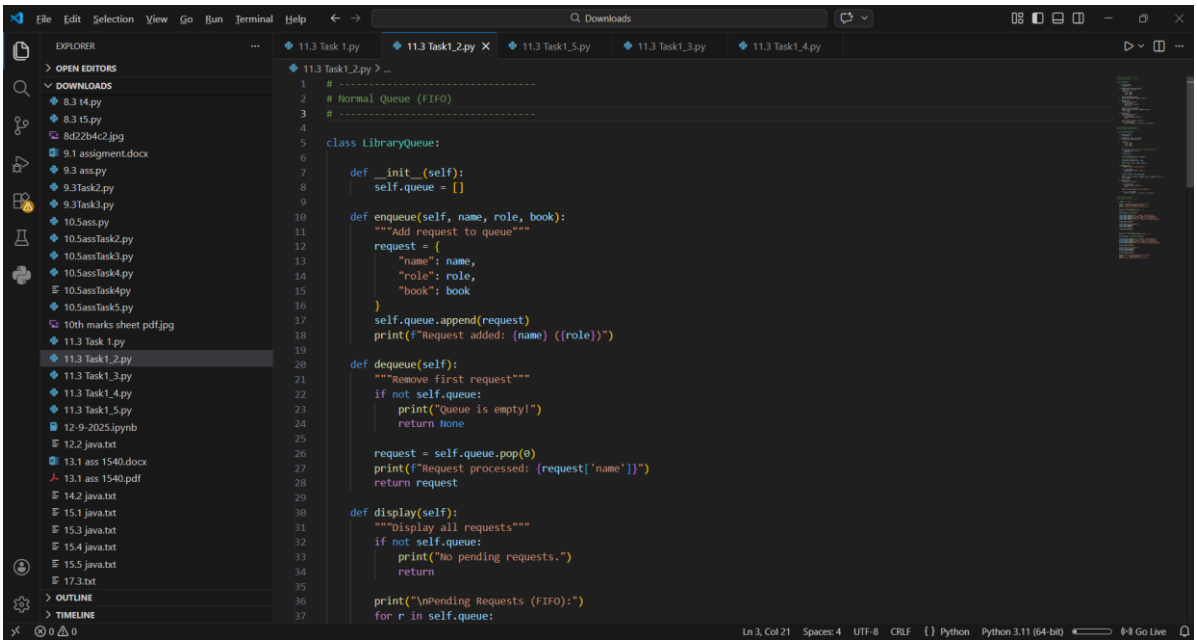
4. Test the system with a mix of student and faculty requests.

Expected Outcome

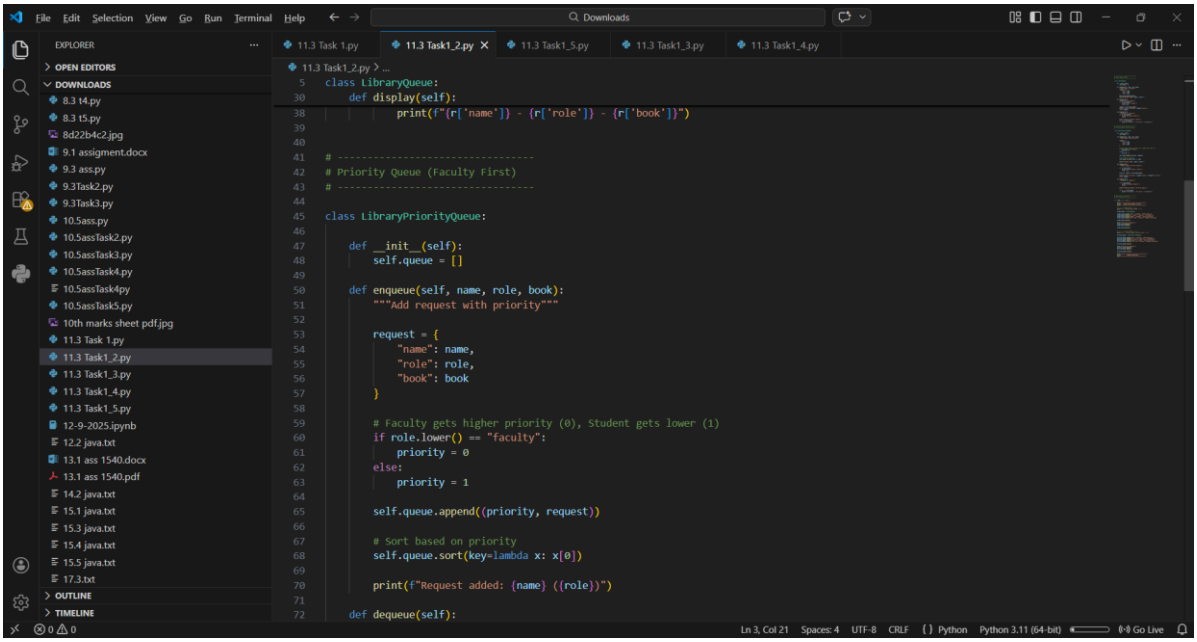
Working queue and priority queue implementations.

• Correct prioritization of faculty requests.

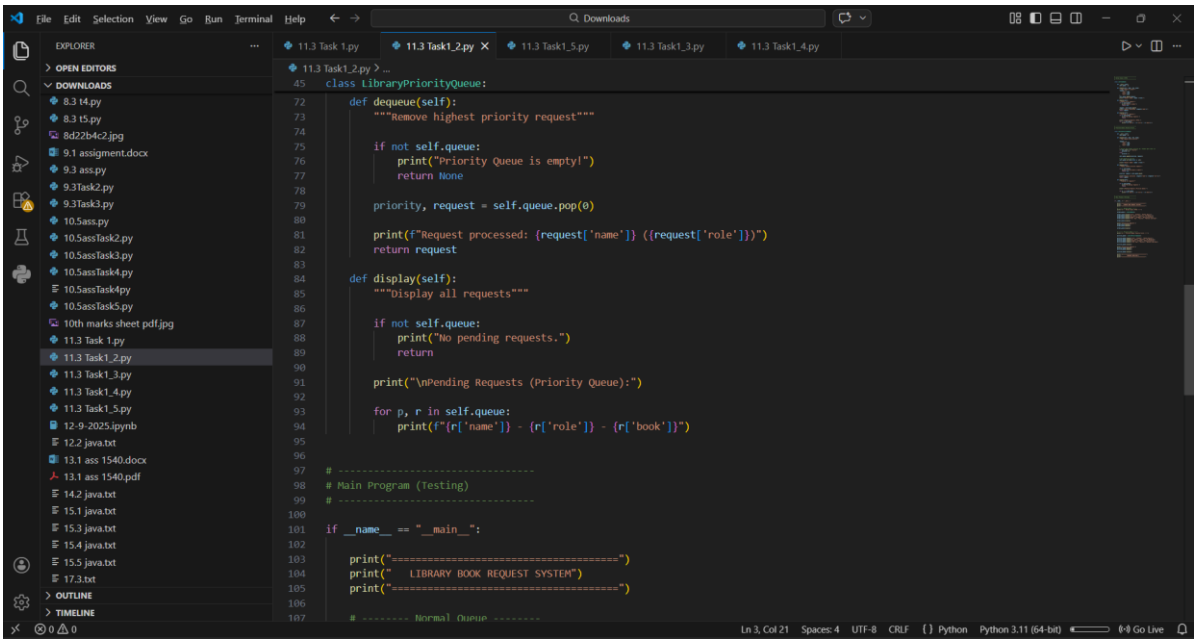
CODE:



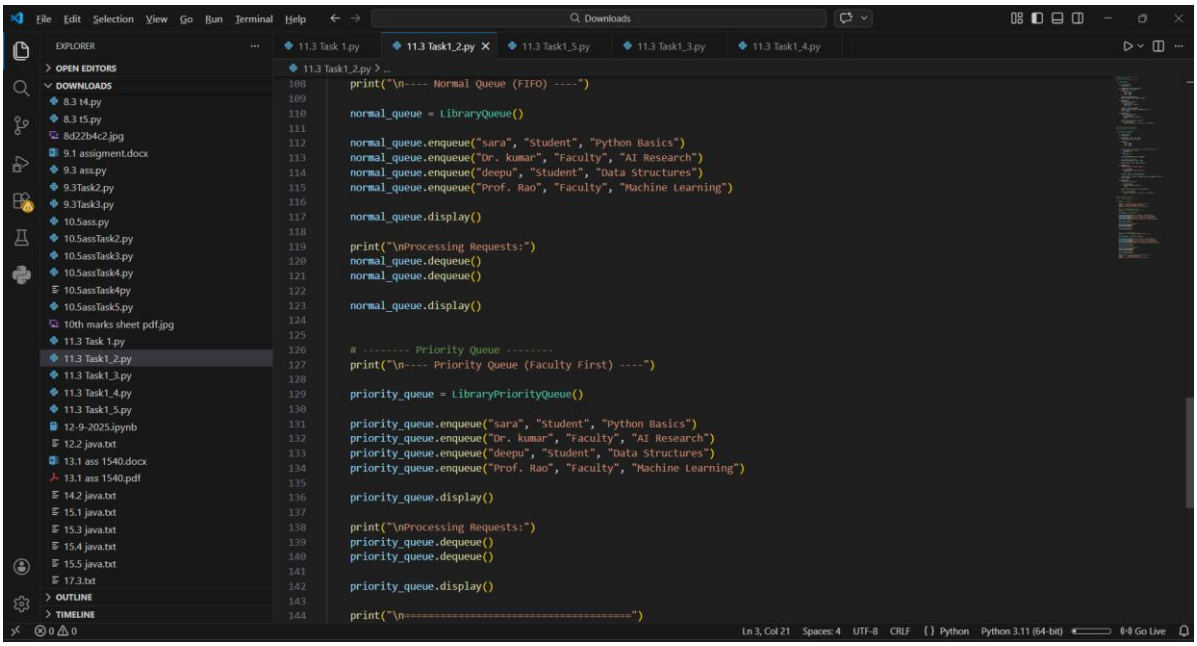
```python
# ---------------------------------
# Normal Queue (FIFO)
# ---------------------------------

class LibraryQueue:

    def __init__(self):
        self.queue = []

    def enqueue(self, name, role, book):
        """Add request to queue"""
        request = {
            "name": name,
            "role": role,
            "book": book
        }
        self.queue.append(request)
        print(f"Request added: {name} ({role})")

    def dequeue(self):
        """Remove first request"""
        if not self.queue:
            print("Queue is empty!")
            return None

        request = self.queue.pop(0)
        print(f"Request processed: {request['name']}")
        return request

    def display(self):
        """Display all requests"""
        if not self.queue:
            print("No pending requests.")
            return

        print("\nPending Requests (FIFO):")
        for r in self.queue:
```



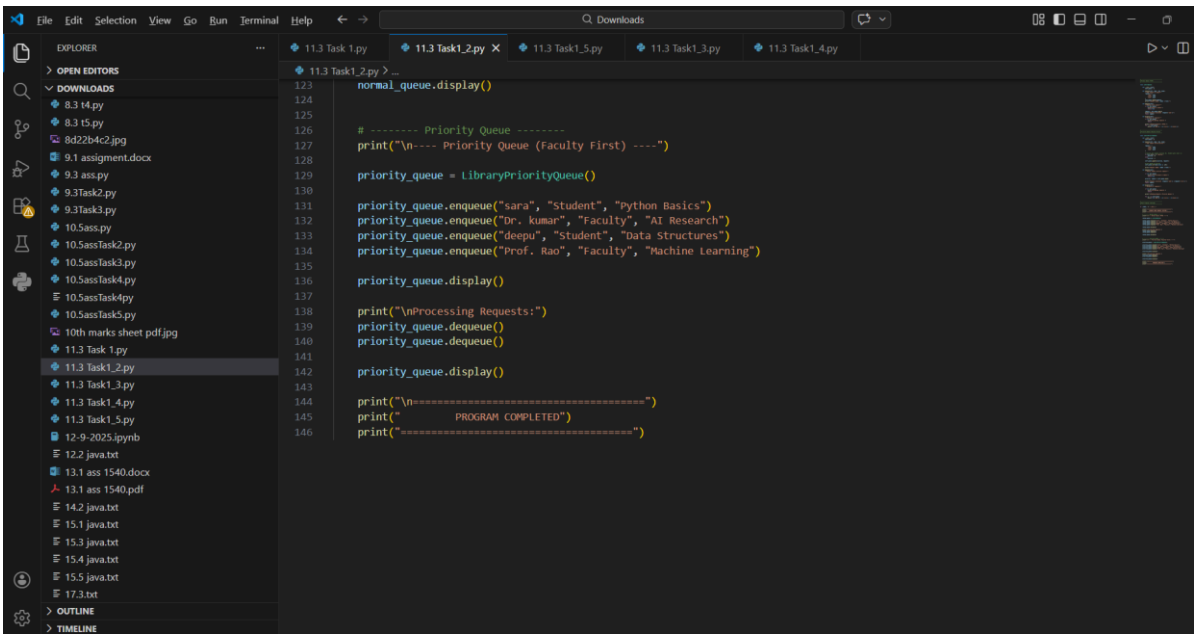
```python
class LibraryQueue:
    def display(self):
            print(f"{r['name']} - {r['role']} - {r['book']}")


# ---------------------------------
# Priority Queue (Faculty First)
# ---------------------------------

class LibraryPriorityQueue:

    def __init__(self):
        self.queue = []

    def enqueue(self, name, role, book):
        """Add request with priority"""

        request = {
            "name": name,
            "role": role,
            "book": book
        }

        # Faculty gets higher priority (0), Student gets lower (1)
        if role.lower() == "faculty":
            priority = 0
        else:
            priority = 1

        self.queue.append((priority, request))

        # Sort based on priority
        self.queue.sort(key=lambda x: x[0])

        print(f"Request added: {name} ({role})")

    def dequeue(self):
```



```
class LibraryPriorityQueue:
    def dequeue(self):
        """Remove highest priority request"""

        if not self.queue:
            print("Priority Queue is empty!")
            return None

        priority, request = self.queue.pop(0)

        print(f"Request processed: {request['name']} ({request['role']})")
        return request

    def display(self):
        """Display all requests"""

        if not self.queue:
            print("No pending requests.")
            return

        print("\nPending Requests (Priority Queue):")

        for p, r in self.queue:
            print(f"{r['name']} - {r['role']} - {r['book']}")


# --------------------------------
# Main Program (Testing)
# --------------------------------

if __name__ == "__main__":

    print("====================================")
    print("   LIBRARY BOOK REQUEST SYSTEM")
    print("====================================")

    # -------- Normal Queue --------
```

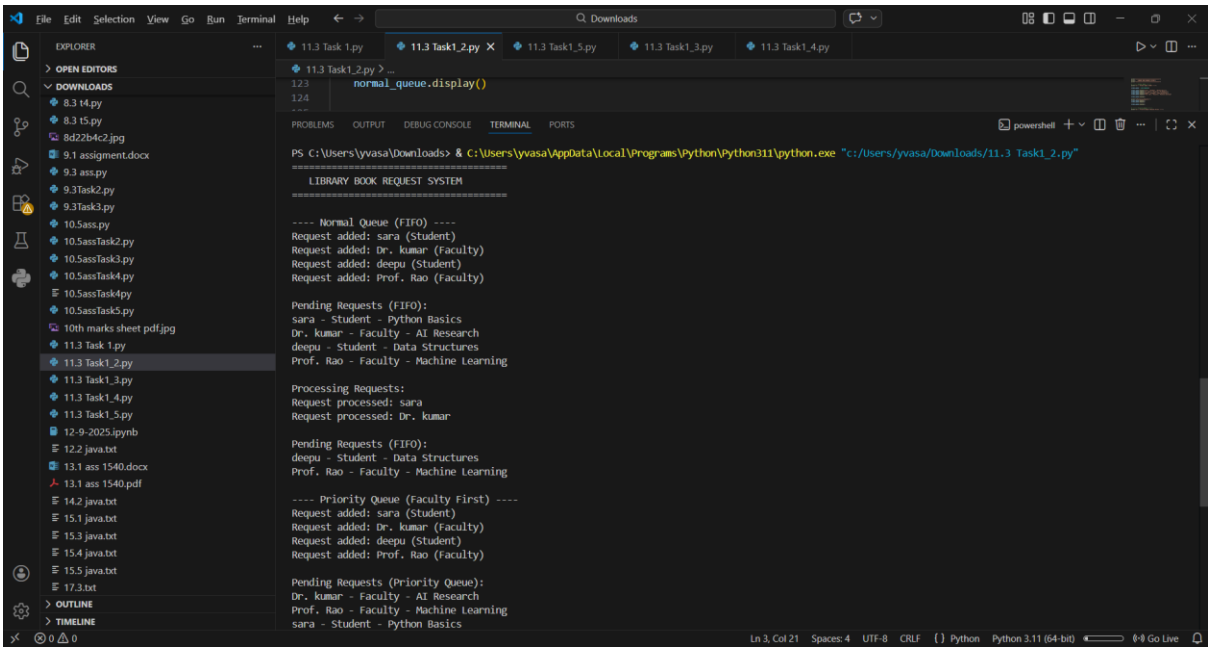


```
    print("\n---- Normal Queue (FIFO) ----")

    normal_queue = LibraryQueue()

    normal_queue.enqueue("sara", "Student", "Python Basics")
    normal_queue.enqueue("Dr. kumar", "Faculty", "AI Research")
    normal_queue.enqueue("deepu", "Student", "Data Structures")
    normal_queue.enqueue("Prof. Rao", "Faculty", "Machine Learning")

    normal_queue.display()

    print("\nProcessing Requests:")
    normal_queue.dequeue()
    normal_queue.dequeue()

    normal_queue.display()


    # -------- Priority Queue --------
    print("\n---- Priority Queue (Faculty First) ----")

    priority_queue = LibraryPriorityQueue()

    priority_queue.enqueue("sara", "Student", "Python Basics")
    priority_queue.enqueue("Dr. kumar", "Faculty", "AI Research")
    priority_queue.enqueue("deepu", "Student", "Data Structures")
    priority_queue.enqueue("Prof. Rao", "Faculty", "Machine Learning")

    priority_queue.display()

    print("\nProcessing Requests:")
    priority_queue.dequeue()
    priority_queue.dequeue()

    priority_queue.display()

    print("\n====================================")
```



```
    normal_queue.display()


    # -------- Priority Queue --------
    print("\n---- Priority Queue (Faculty First) ----")

    priority_queue = LibraryPriorityQueue()

    priority_queue.enqueue("sara", "Student", "Python Basics")
    priority_queue.enqueue("Dr. kumar", "Faculty", "AI Research")
    priority_queue.enqueue("deepu", "Student", "Data Structures")
    priority_queue.enqueue("Prof. Rao", "Faculty", "Machine Learning")

    priority_queue.display()

    print("\nProcessing Requests:")
    priority_queue.dequeue()
    priority_queue.dequeue()

    priority_queue.display()

    print("\n====================================")
    print("          PROGRAM COMPLETED")
    print("====================================")
```

OUTPUT:


File Edit Selection View Go Run Terminal Help
Downloads
EXPLORER
OPEN EDITORS
DOWNLOADS
11.3 Task 1.py
11.3 Task1_2.py
11.3 Task1_5.py
11.3 Task1_3.py
11.3 Task1_4.py
normal_queue.display()
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
powershell
PS C:\Users\yvasa\Downloads> & C:\Users\yvasa\AppData\Local\Programs\Python\Python311\python.exe "c:/Users/yvasa/Downloads/11.3 Task1_2.py"
LIBRARY BOOK REQUEST SYSTEM
---- Normal Queue (FIFO) ----
Request added: sara (Student)
Request added: Dr. kumar (Faculty)
Request added: deepu (Student)
Request added: Prof. Rao (Faculty)
Pending Requests (FIFO):
sara - Student - Python Basics
Dr. kumar - Faculty - AI Research
deepu - Student - Data Structures
Prof. Rao - Faculty - Machine Learning
Processing Requests:
Request processed: sara
Request processed: Dr. kumar
Pending Requests (FIFO):
deepu - Student - Data Structures
Prof. Rao - Faculty - Machine Learning
---- Priority Queue (Faculty First) ----
Request added: sara (Student)
Request added: Dr. kumar (Faculty)
Request added: deepu (Student)
Request added: Prof. Rao (Faculty)
Pending Requests (Priority Queue):
Dr. kumar - Faculty - AI Research
Prof. Rao - Faculty - Machine Learning
sara - Student - Python Basics
Ln 3, Col 21 Spaces: 4 UTF-8 CRLF Python Python 3.11 (64-bit) Go Live

Task 3: Emergency Help Desk (Stack Implementation)

Scenario

SR University's IT Help Desk receives technical support tickets from students and staff.

While tickets are received sequentially, issue escalation follows a

Last-In, First-Out (LIFO) approach.

Tasks

1. Implement a Stack to manage support tickets.

2. Provide the following operations:

o push(ticket) o

pop() o peek()

3. Simulate at least five tickets being raised and resolved.

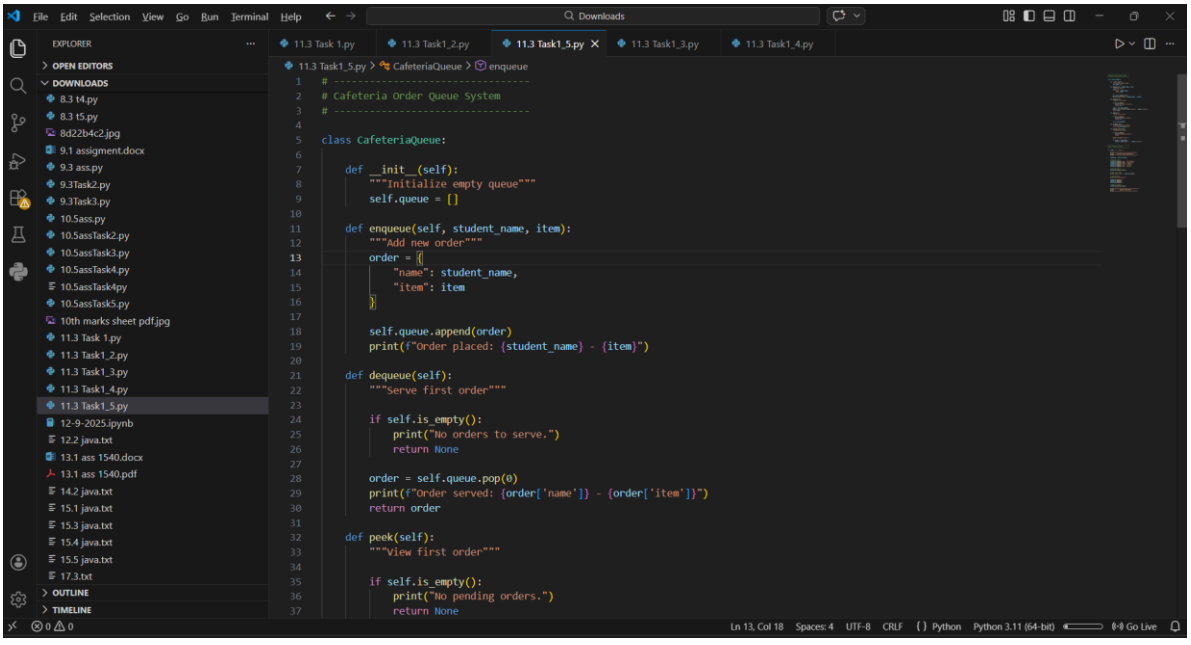
4. Use GitHub Copilot to suggest additional stack operations such as: o Checking

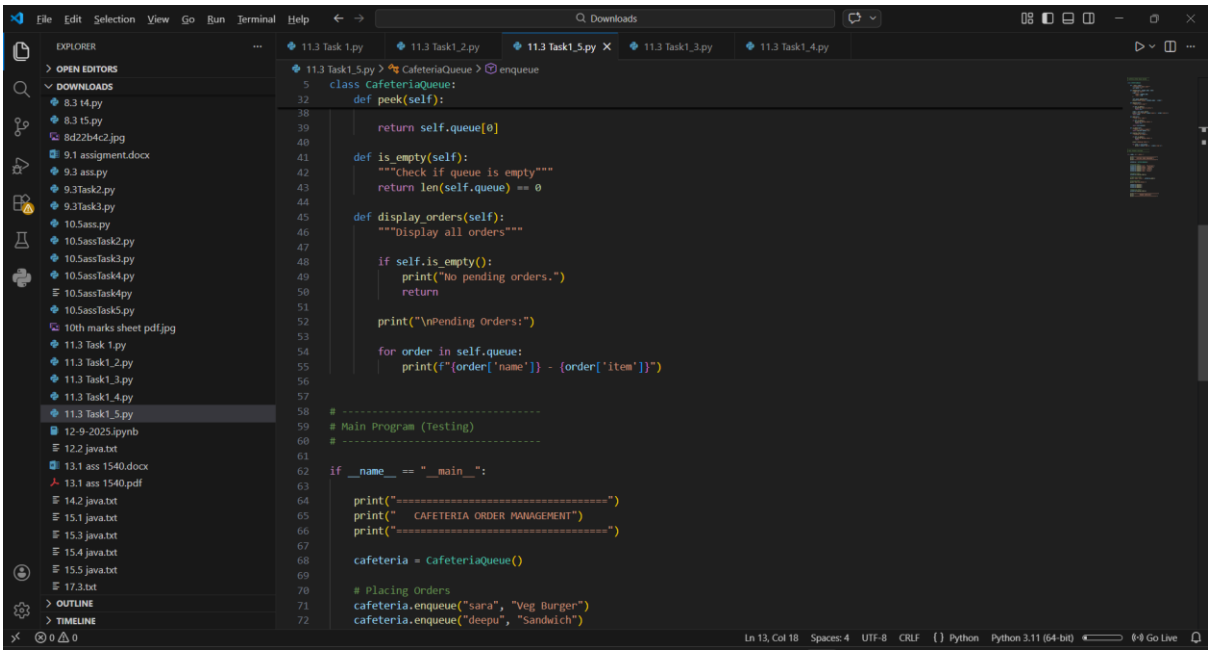
   whether the stack is empty o Checking whether the stack is full (if applicable)

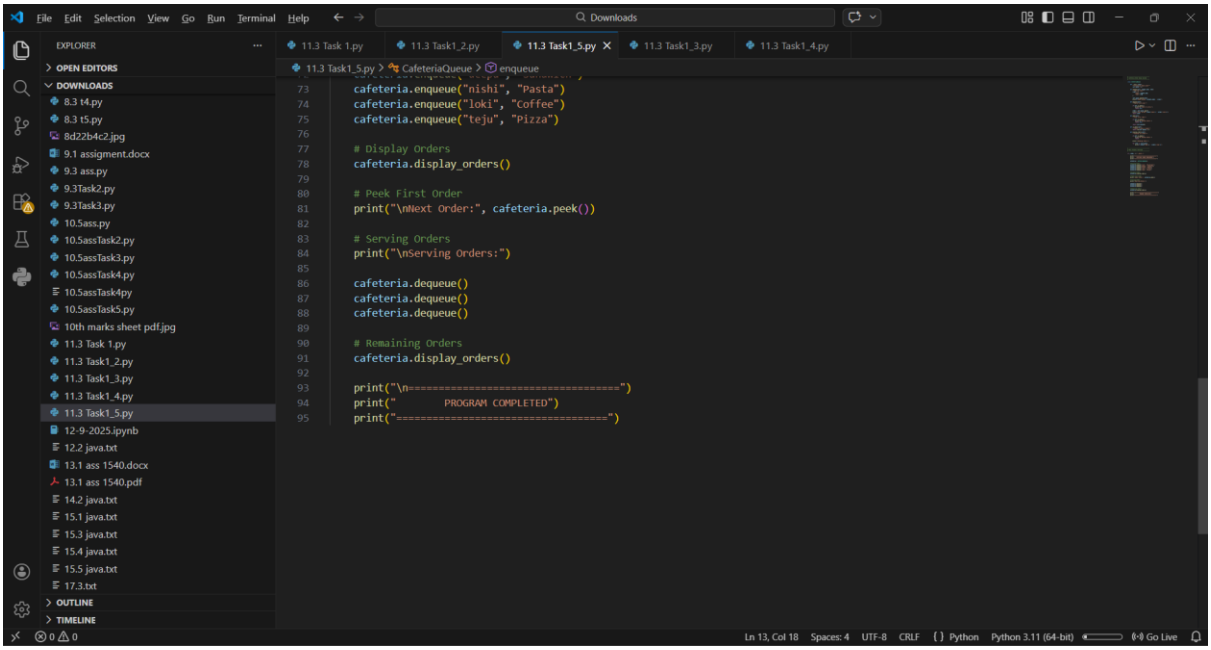
Expected Outcome

- Functional stack-based ticket management system.
- Clear demonstration of LIFO behavior.

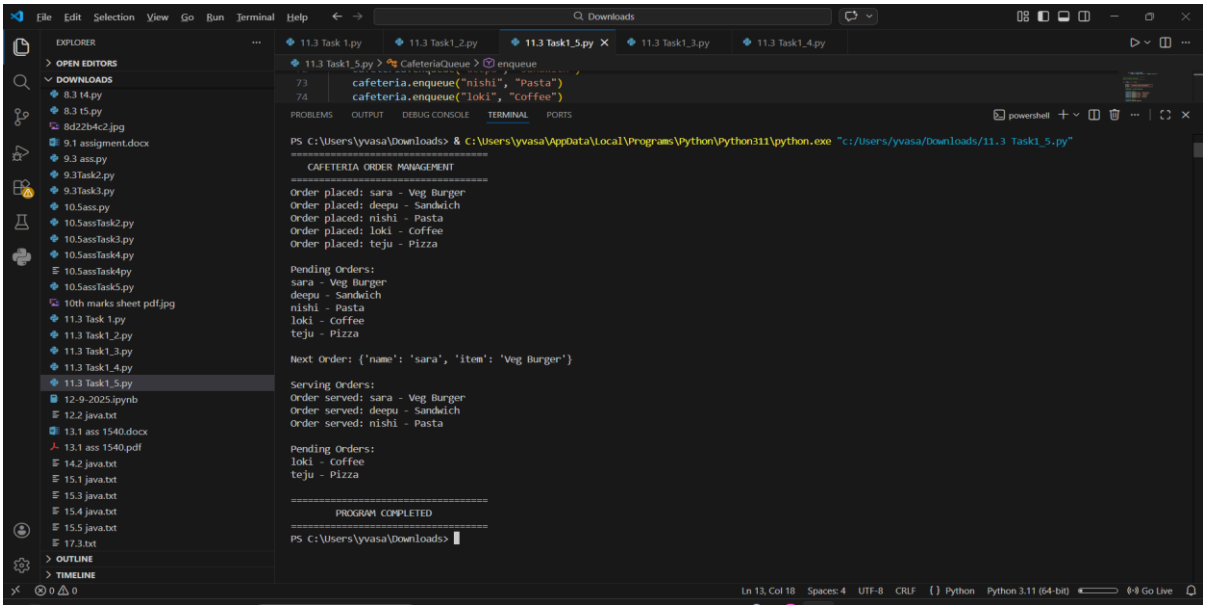
CODE:




File Edit Selection View Go Run Terminal Help
Downloads
EXPLORER
OPEN EDITORS
DOWNLOADS
11.3 Task 1.py
11.3 Task1_2.py
11.3 Task1_5.py
11.3 Task1_3.py
11.3 Task1_4.py
11.3 Task1_5.py > CafeteriaQueue > enqueue
class CafeteriaQueue:
def peek(self):
return self.queue[0]
def is_empty(self):
"""Check if queue is empty"""
return len(self.queue) == 0
def display_orders(self):
"""Display all orders"""
if self.is_empty():
print("No pending orders.")
return
print("\nPending Orders:")
for order in self.queue:
print(f"{order['name']} - {order['item']}")
# --------------------------------
# Main Program (Testing)
# --------------------------------
if __name__ == "__main__":
print("==================================")
print("   CAFETERIA ORDER MANAGEMENT")
print("==================================")
cafeteria = CafeteriaQueue()
# Placing Orders
cafeteria.enqueue("sara", "Veg Burger")
cafeteria.enqueue("deepu", "Sandwich")
Ln 13, Col 18 Spaces: 4 UTF-8 CRLF Python Python 3.11 (64-bit) Go Live


File Edit Selection View Go Run Terminal Help
Downloads
EXPLORER
OPEN EDITORS
DOWNLOADS
11.3 Task 1.py
11.3 Task1_2.py
11.3 Task1_5.py
11.3 Task1_3.py
11.3 Task1_4.py
11.3 Task1_5.py > CafeteriaQueue > enqueue
cafeteria.enqueue("nishi", "Pasta")
cafeteria.enqueue("loki", "Coffee")
cafeteria.enqueue("teju", "Pizza")
# Display Orders
cafeteria.display_orders()
# Peek First Order
print("\nNext Order:", cafeteria.peek())
# Serving Orders
print("\nServing Orders:")
cafeteria.dequeue()
cafeteria.dequeue()
cafeteria.dequeue()
# Remaining Orders
cafeteria.display_orders()
print("\n==================================")
print("   PROGRAM COMPLETED")
print("==================================")
Ln 13, Col 18 Spaces: 4 UTF-8 CRLF Python Python 3.11 (64-bit) Go Live

OUTPUT:



Task 4: Hash Table

Objective

To implement a Hash Table and understand collision handling.

Task Description

Use AI to generate a hash table with:

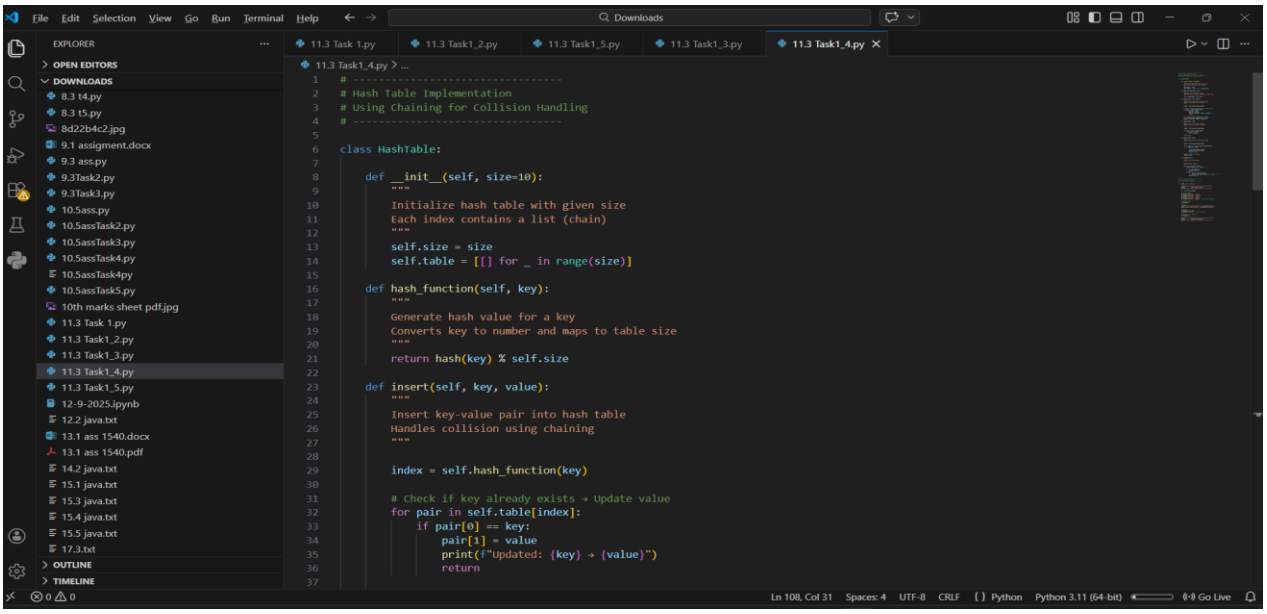
- Insert
- Search
- DeleteStarter Code class HashTable:

pass

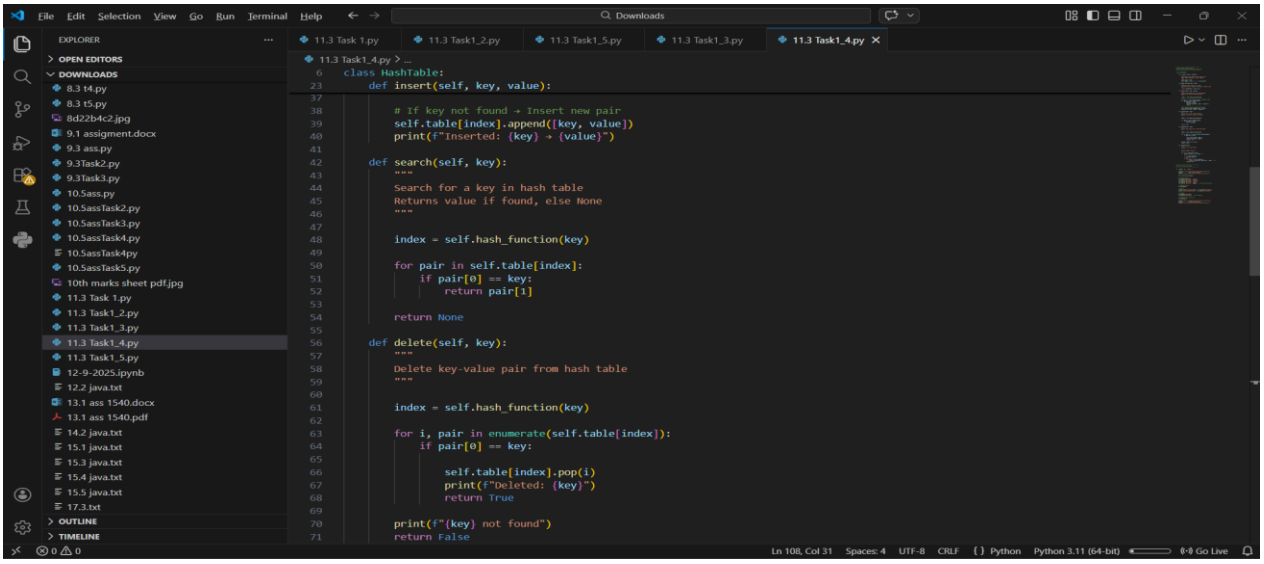
Expected Outcome

- Collision handling using chaining
- Well-commented methods

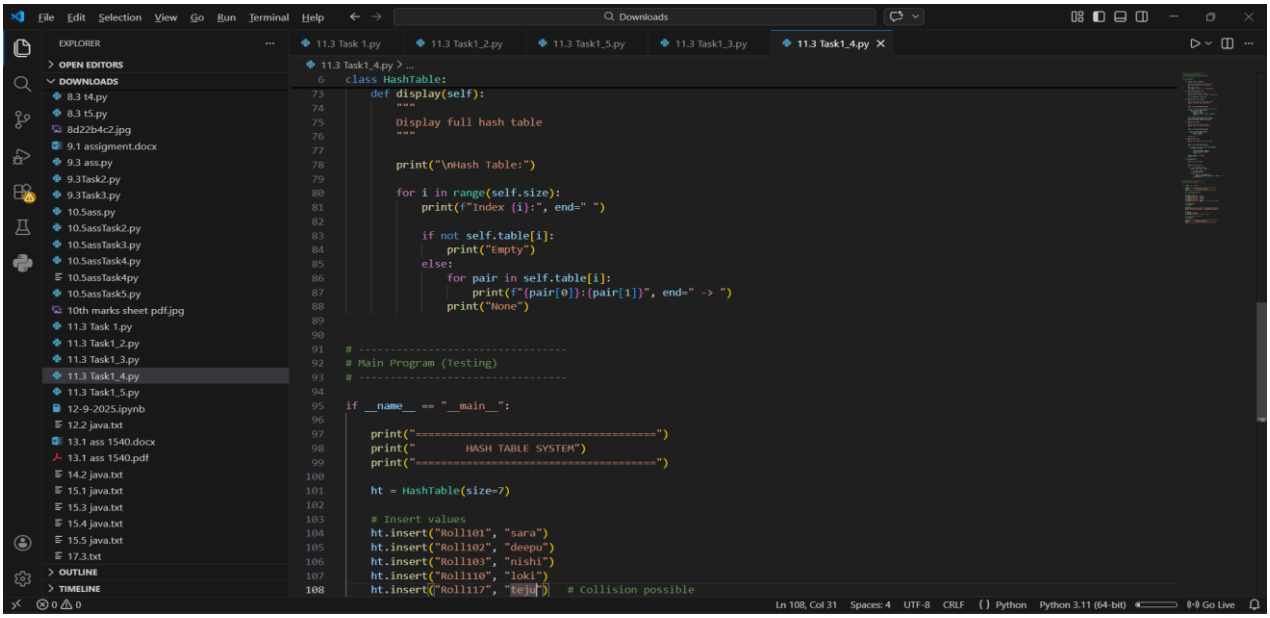
CODE:



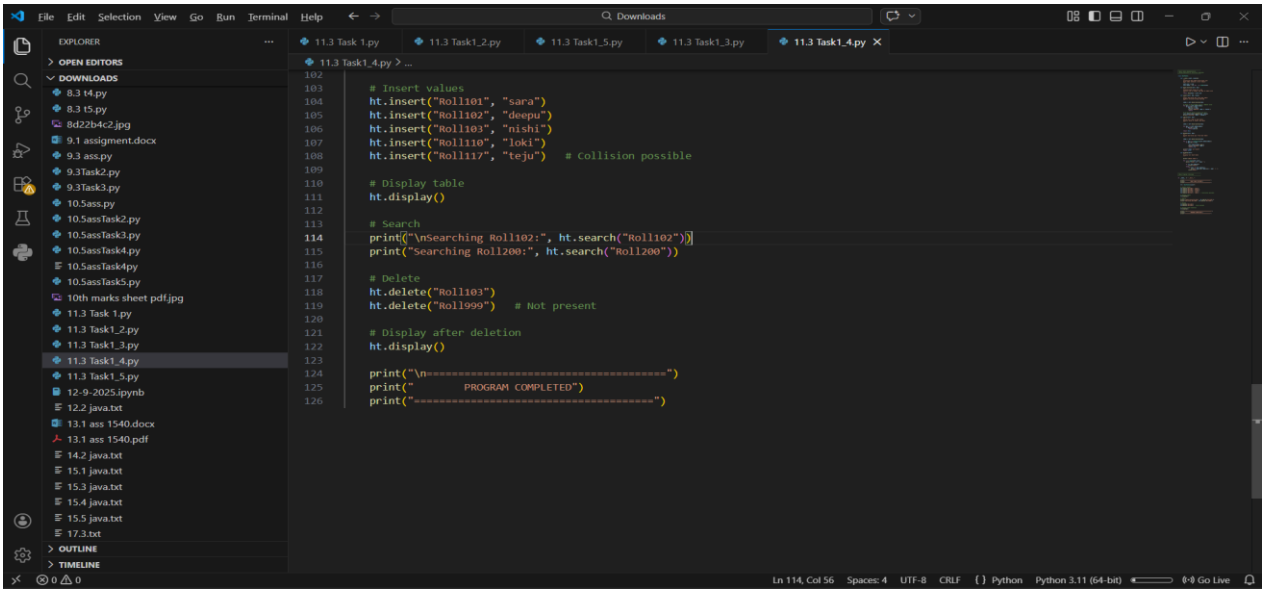
```python
# ---------------------------------
# Hash Table Implementation
# Using Chaining for Collision Handling
# ---------------------------------

class HashTable:

    def __init__(self, size=10):
        """
        Initialize hash table with given size
        Each index contains a list (chain)
        """
        self.size = size
        self.table = [[] for _ in range(size)]

    def hash_function(self, key):
        """
        Generate hash value for a key
        Converts key to number and maps to table size
        """
        return hash(key) % self.size

    def insert(self, key, value):
        """
        Insert key-value pair into hash table
        Handles collision using chaining
        """

        index = self.hash_function(key)

        # Check if key already exists → Update value
        for pair in self.table[index]:
            if pair[0] == key:
                pair[1] = value
                print(f"Updated: {key} → {value}")
                return
```



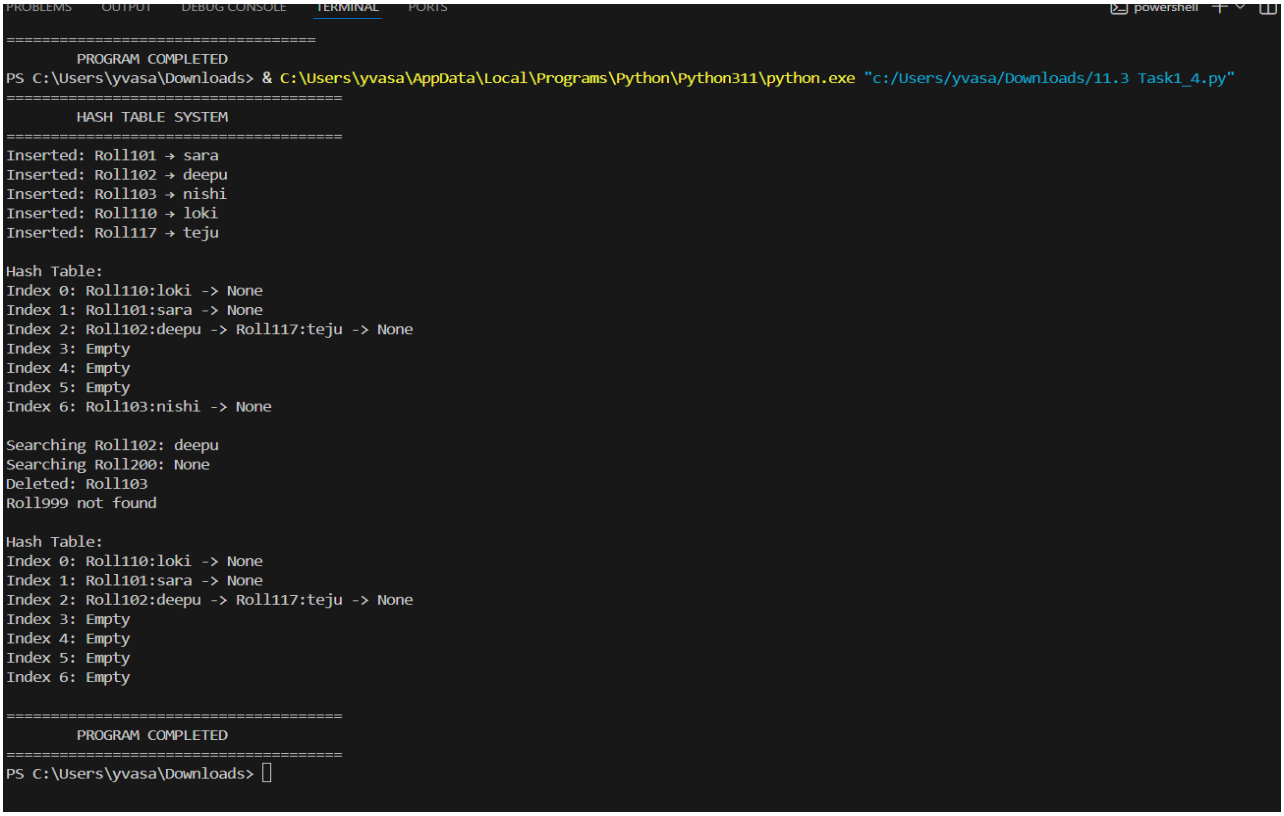
```python
class HashTable:
    def insert(self, key, value):

        # If key not found → Insert new pair
        self.table[index].append([key, value])
        print(f"Inserted: {key} → {value}")

    def search(self, key):
        """
        Search for a key in hash table
        Returns value if found, else None
        """

        index = self.hash_function(key)

        for pair in self.table[index]:
            if pair[0] == key:
                return pair[1]

        return None

    def delete(self, key):
        """
        Delete key-value pair from hash table
        """

        index = self.hash_function(key)

        for i, pair in enumerate(self.table[index]):
            if pair[0] == key:

                self.table[index].pop(i)
                print(f"Deleted: {key}")
                return True

        print(f"{key} not found")
        return False
```



```python
class HashTable:
    def display(self):
        """
        Display full hash table
        """

        print("\nHash Table:")

        for i in range(self.size):
            print(f"Index {i}:", end=" ")

            if not self.table[i]:
                print("Empty")
            else:
                for pair in self.table[i]:
                    print(f"{pair[0]}:{pair[1]}", end=" -> ")
                print("None")


# ---------------------------------
# Main Program (Testing)
# ---------------------------------

if __name__ == "__main__":

    print("====================================")
    print("        HASH TABLE SYSTEM")
    print("====================================")

    ht = HashTable(size=7)

    # Insert values
    ht.insert("Roll101", "sara")
    ht.insert("Roll102", "deepu")
    ht.insert("Roll103", "nishi")
    ht.insert("Roll110", "loki")
    ht.insert("Roll117", "teju")  # Collision possible
```


File Edit Selection View Go Run Terminal Help
Downloads
EXPLORER
OPEN EDITORS
DOWNLOADS
11.3 Task 1.py
11.3 Task1_2.py
11.3 Task1_5.py
11.3 Task1_3.py
11.3 Task1_4.py
11.3 Task1_4.py > ...
# Insert values
ht.insert("Roll101", "sara")
ht.insert("Roll102", "deepu")
ht.insert("Roll103", "nishi")
ht.insert("Roll110", "loki")
ht.insert("Roll117", "teju") # Collision possible
# Display table
ht.display()
# Search
print("\nSearching Roll102:", ht.search("Roll102"))
print("Searching Roll200:", ht.search("Roll200"))
# Delete
ht.delete("Roll103")
ht.delete("Roll999") # Not present
# Display after deletion
ht.display()
print("\n====================================")
print("   PROGRAM COMPLETED")
print("====================================")
OUTLINE
TIMELINE
Ln 114, Col 56 Spaces: 4 UTF-8 CRLF Python Python 3.11 (64-bit) Go Live

OUTPUT:


PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
powershell
====================================
PROGRAM COMPLETED
PS C:\Users\yvasa\Downloads> & C:\Users\yvasa\AppData\Local\Programs\Python\Python311\python.exe "c:/Users/yvasa/Downloads/11.3 Task1_4.py"
====================================
HASH TABLE SYSTEM
====================================
Inserted: Roll101 → sara
Inserted: Roll102 → deepu
Inserted: Roll103 → nishi
Inserted: Roll110 → loki
Inserted: Roll117 → teju
Hash Table:
Index 0: Roll110:loki -> None
Index 1: Roll101:sara -> None
Index 2: Roll102:deepu -> Roll117:teju -> None
Index 3: Empty
Index 4: Empty
Index 5: Empty
Index 6: Roll103:nishi -> None
Searching Roll102: deepu
Searching Roll200: None
Deleted: Roll103
Roll999 not found
Hash Table:
Index 0: Roll110:loki -> None
Index 1: Roll101:sara -> None
Index 2: Roll102:deepu -> Roll117:teju -> None
Index 3: Empty
Index 4: Empty
Index 5: Empty
Index 6: Empty
====================================
PROGRAM COMPLETED
====================================
PS C:\Users\yvasa\Downloads>

Task 5: Real-Time Application Challenge

Scenario

Design a Campus Resource Management System with the following features:

- Student Attendance Tracking
- Event Registration System
- Library Book Borrowing
- Bus Scheduling System
- Cafeteria Order Queue

Student Tasks

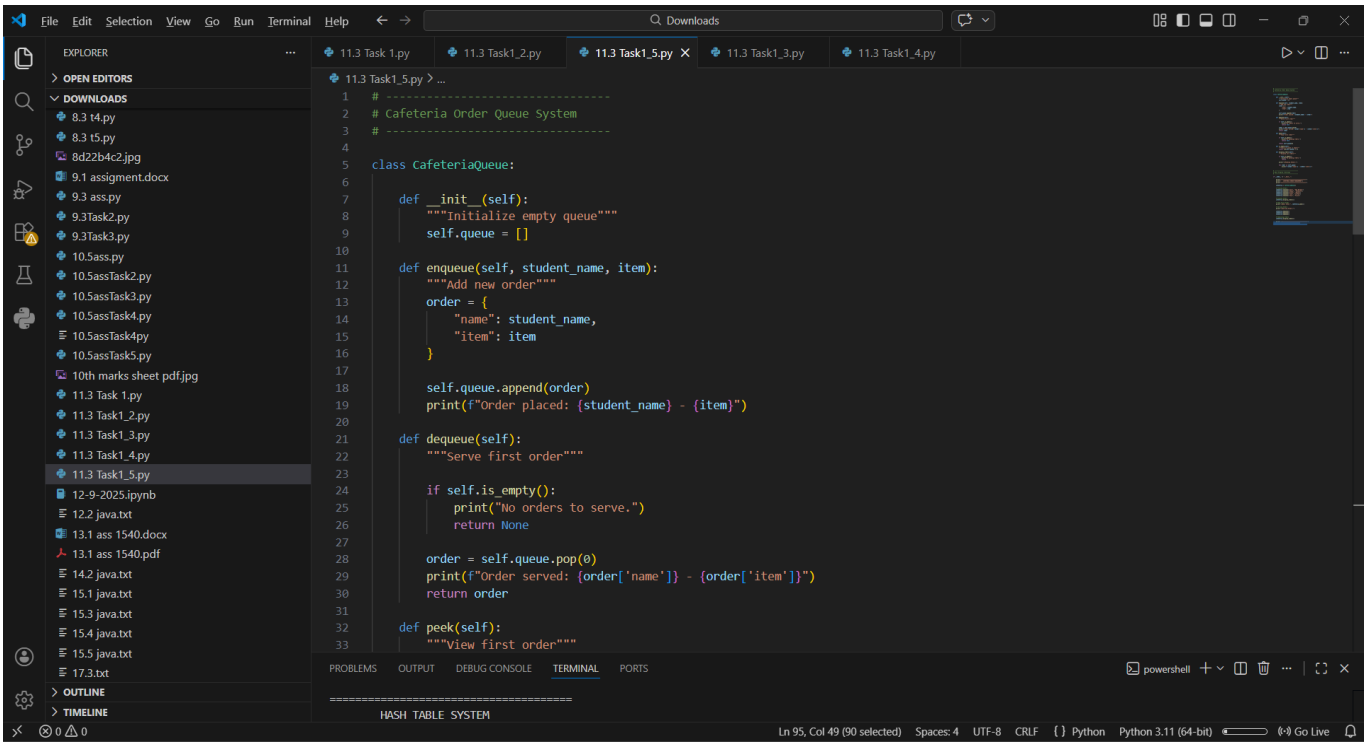
1. Choose the most appropriate data structure for each feature.
2. Justify your choice in 2–3 sentences.
3. Implement one selected feature using AI-assisted code generation.

Expected Outcome

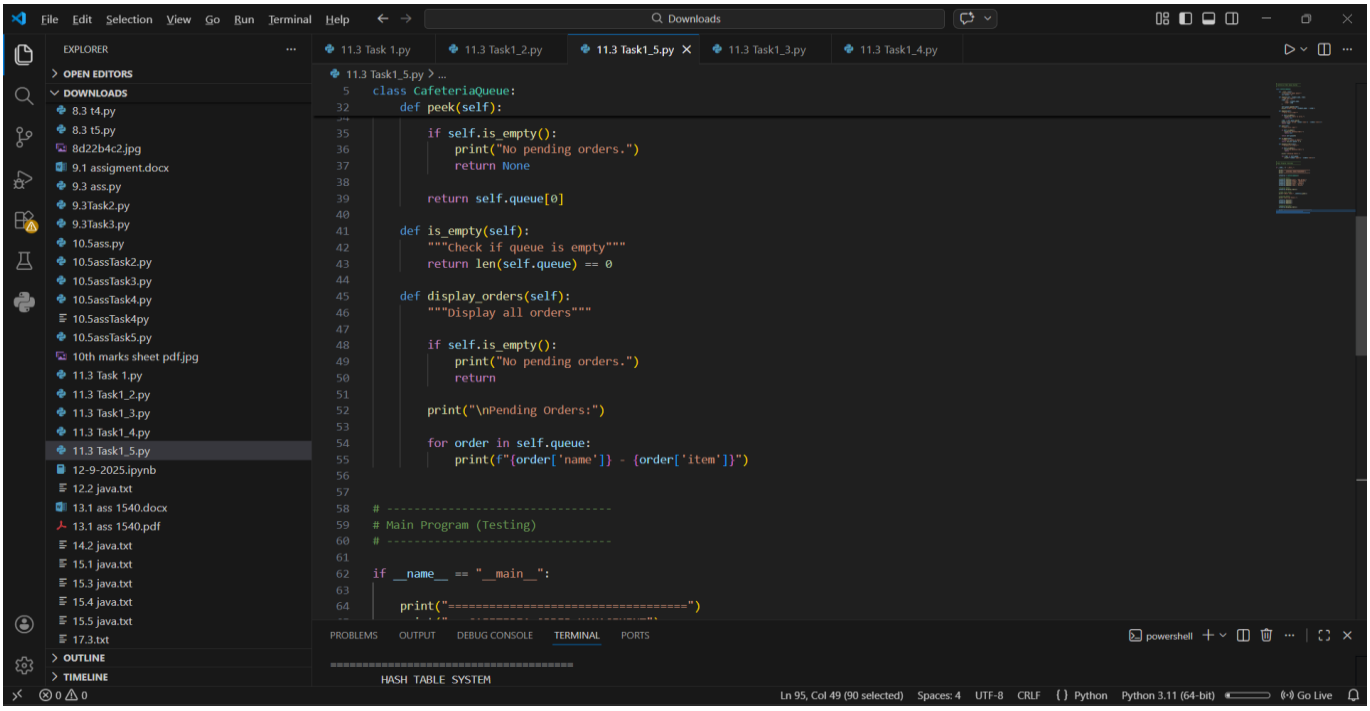
- Mapping table: Feature → Data Structure → Justification
- One fully working Python implementation

Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.

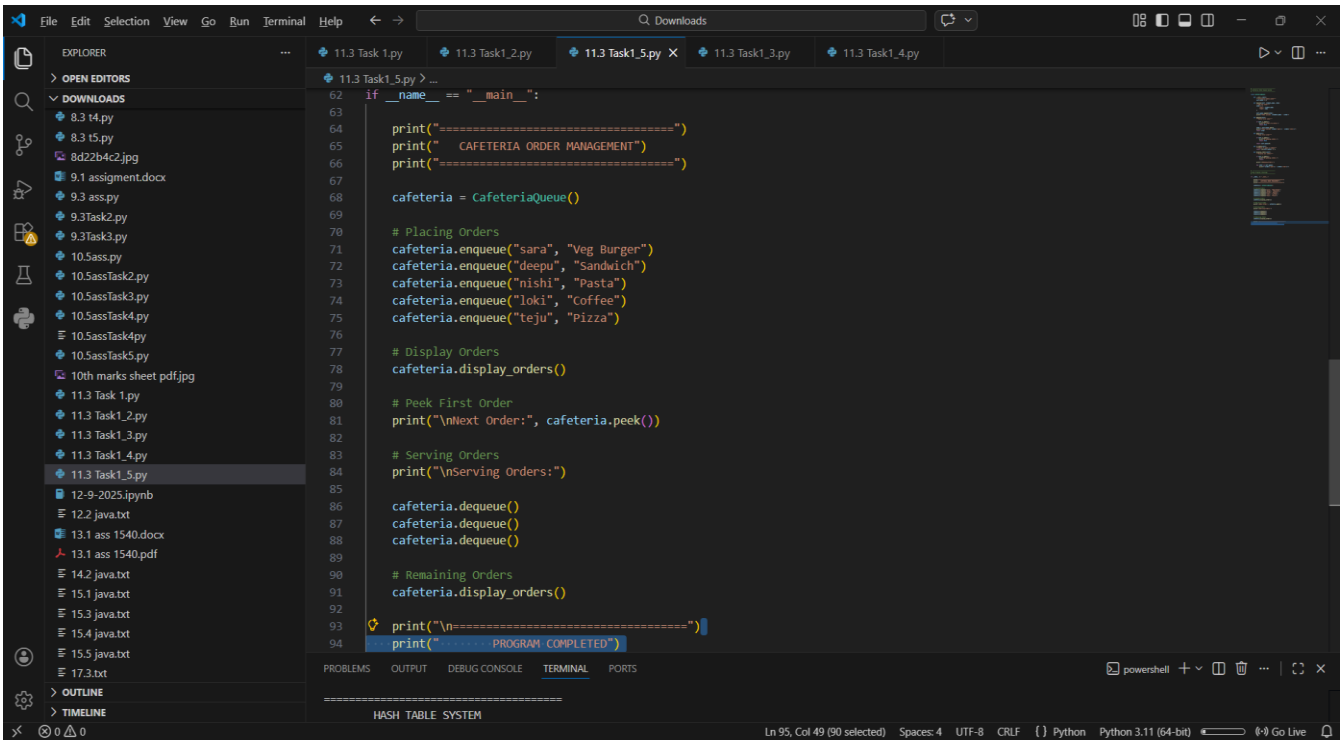
CODE:



```
# ---------------------------------
# Cafeteria Order Queue System
# ---------------------------------

class CafeteriaQueue:

    def __init__(self):
        """Initialize empty queue"""
        self.queue = []

    def enqueue(self, student_name, item):
        """Add new order"""
        order = {
            "name": student_name,
            "item": item
        }

        self.queue.append(order)
        print(f"Order placed: {student_name} - {item}")

    def dequeue(self):
        """Serve first order"""

        if self.is_empty():
            print("No orders to serve.")
            return None

        order = self.queue.pop(0)
        print(f"Order served: {order['name']} - {order['item']}")
        return order

    def peek(self):
        """View first order"""
```



```
class CafeteriaQueue:
    def peek(self):

        if self.is_empty():
            print("No pending orders.")
            return None

        return self.queue[0]

    def is_empty(self):
        """Check if queue is empty"""
        return len(self.queue) == 0

    def display_orders(self):
        """Display all orders"""

        if self.is_empty():
            print("No pending orders.")
            return

        print("\nPending Orders:")

        for order in self.queue:
            print(f"{order['name']} - {order['item']}")


# ---------------------------------
# Main Program (Testing)
# ---------------------------------

if __name__ == "__main__":

    print("==================================")
```




```
    cafeteria.display_orders()

    # Peek First Order
    print("\nNext Order:", cafeteria.peek())

    # Serving Orders
    print("\nServing Orders:")

    cafeteria.dequeue()
    cafeteria.dequeue()
    cafeteria.dequeue()

    # Remaining Orders
    cafeteria.display_orders()

    print("\n==================================")
    print("        PROGRAM COMPLETED")
    print("==================================")
```

OUTPUT:

```
68 |     cafeteria = CafeteriaQueue()
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell

```
        PROGRAM COMPLETED
==================================
PS C:\Users\yvasa\Downloads> & C:\Users\yvasa\AppData\Local\Programs\Python\Python311\python.exe "c:/Users/yvasa/Downloads/11.3 Task1_5.py"
==================================
   CAFETERIA ORDER MANAGEMENT
==================================
Order placed: sara - Veg Burger
Order placed: deepu - Sandwich
Order placed: nishi - Pasta
Order placed: loki - Coffee
Order placed: teju - Pizza

Pending Orders:
sara - Veg Burger
deepu - Sandwich
nishi - Pasta
loki - Coffee
teju - Pizza

Next Order: {'name': 'sara', 'item': 'Veg Burger'}

Serving Orders:
Order served: sara - Veg Burger
Order served: deepu - Sandwich
Order served: nishi - Pasta

Pending Orders:
loki - Coffee
teju - Pizza

==================================
        PROGRAM COMPLETED
==================================
PS C:\Users\yvasa\Downloads>
```

OUTPUT: